	Pimpri-Chinchwad Education Trust's **Pimpri-Chinchwad College of Engineering** (An Autonomous Institute) Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

Assignment No 1

Name: Soham Kulkarni

PRN: 124B1F056

Subject: Design and Analysis of Algorithms

Batch: A3

GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Design and implement a sorting algorithm using Merge Sort to arrange warehouse inventory records in ascending order of their stock quantity. The implementation must scale comfortably to large datasets (on the order of a million records) without a significant rise in computation time. The assignment also studies the time complexity of Merge Sort and compares it against conventional sorting techniques such as Bubble Sort and Insertion Sort.

2. Course Objective

- To understand the fundamentals of computational complexity across different algorithms.
- To choose an appropriate algorithm design strategy for solving practical, real-world problems.

3. Course Outcome

- Analyze the asymptotic behaviour and performance of algorithms.
- Apply suitable Divide-and-Conquer or Greedy paradigms to solve computational problems.

Theory

Merge Sort is a comparison-based sorting technique that relies on the Divide-and-Conquer strategy. Rather than sorting the entire dataset in one pass, it breaks the problem into smaller, identical sub-problems, solves each of them independently, and then combines the results. Because it processes data in a predictable, systematic manner, Merge Sort is widely regarded as one of the most reliable general-purpose sorting algorithms, and it is also stable, meaning equal elements retain their original relative order after sorting.

Working of Merge Sort

- Divide: The array is repeatedly split into two roughly equal halves until every part contains a single element.

- Conquer: Each of these single-element (trivially sorted) parts is treated as a sorted sub-array.
- Combine: Adjacent sorted sub-arrays are merged pairwise, comparing front elements at each step, until one fully sorted array remains.

Complexity Analysis

- Division step: Splitting a dataset of n elements in half repeatedly takes $\log_2(n)$ levels, since the size is halved at every step.
- Merge step: At every level of recursion, merging touches all n elements exactly once.
- Overall work: (number of levels) × (work per level) = $\log_2(n) \times n$, giving a total running time of O(n log n).

Time Complexity

Best Case

Merge Sort does not check whether the input is already sorted before recursing, so it still performs the full divide and merge sequence regardless of the initial arrangement.

Best Case = O(n log n)

Average Case

For a randomly ordered dataset, the algorithm still divides the array into halves at every level and merges them back; on average, the number of comparisons during each merge stays proportional to n.

Average Case = O(n log n)

Worst Case

Even when the input is arranged in the least favourable order (for example, fully reverse sorted), the same divide-and-merge procedure is followed, so the number of comparisons is at its maximum but still bounded by n log n.

Worst Case = O(n log n)

Space Complexity

During merging, elements are copied into a temporary auxiliary array whose size equals the number of elements being merged at that level. Across the recursion this auxiliary storage requirement is proportional to n, giving Merge Sort a space complexity of O(n).

Merge Sort Algorithm — Steps

- Step 1: If the sub-list contains only one element, it is already sorted; return immediately.
- Step 2: Otherwise, split the list into two halves and recursively sort each half.
- Step 3: Merge the two sorted halves into a single sorted list.

Pseudocode

```
ALGORITHM MergeSort(A, start, end)
1. if start >= end then
2.     return
```

```
3. end if
4. mid <- (start + end) / 2
5. MergeSort(A, start, mid)
6. MergeSort(A, mid + 1, end)
7. Merge(A, start, mid, end)
END ALGORITHM

ALGORITHM Merge(A, start, mid, end)
1. Create empty array TEMP
2. p <- start
3. q <- mid + 1
4. while p <= mid AND q <= end do
5.     if A[p] <= A[q] then
6.         Append A[p] to TEMP; p <- p + 1
7.     else
8.         Append A[q] to TEMP; q <- q + 1
9.     end if
10. end while
11. while p <= mid do
12.     Append A[p] to TEMP; p <- p + 1
13. end while
14. while q <= end do
15.     Append A[q] to TEMP; q <- q + 1
16. end while
17. Copy TEMP back into A[start...end]
END ALGORITHM
```

Flow of Execution

- START -> Read the array of inventory stock quantities.
- Check if the current sub-array has only one element; if yes, it is already sorted, return it.
- If not, divide the sub-array into two halves and recursively call Merge Sort on each half.
- Compare the front elements of the two sorted halves one at a time and copy the smaller value into the result array.
- Repeat the comparison-and-copy step until every element from both halves has been placed.
- Return the merged, sorted sub-array; repeat this merging at every level until the whole array is sorted.
- END -> Output the inventory records sorted by stock quantity.

6. Example

Warehouse stock quantities: [45, 12, 78, 23]

Step 1: Divide

- Split into halves -> [45, 12] and [78, 23]
- Split further -> [45], [12], [78], [23]

Step 2: Merge & Sort

- Merge [45] and [12] -> [12, 45]
- Merge [78] and [23] -> [23, 78]
- Merge [12, 45] and [23, 78] -> [12, 23, 45, 78]

Output:

Stock quantities sorted -> [12, 23, 45, 78]

Conclusion

Merge Sort proves to be a dependable and stable technique for arranging inventory records by stock quantity, consistently delivering O(n log n) performance regardless of how the input data is arranged. Unlike quadratic-time methods such as Bubble Sort or Insertion Sort, which become impractical as data volume grows, Merge Sort scales gracefully to very large datasets, making it well suited for real-world systems such as inventory management, warehouse automation, and large-scale record keeping.